



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: __Pablo David Molina Son____ **Carné:** __1338825____

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Una API REST es una forma estándar de diseñar servicios web para que diferentes programas hablen entre sí usando HTTP. Su regla principal es que no guarda estado: cada petición que envías debe llevar toda la información necesaria para procesarse, sin depender de que el servidor recuerde lo que hiciste antes.

2. ¿Qué es un endpoint?

Un endpoint es la ruta o punto de acceso específico dentro del servidor al que apuntas para interactuar con un recurso. Le indica a la API a qué función o dato exacto quieres llegar.

3. ¿Qué diferencia existe entre una URL y un endpoint?

URL: Es la dirección web completa que te lleva al servidor.

Endpoint: Es el destino lógico dentro de esa dirección junto con la acción que vas a realizar.

Una misma dirección base puede tener varios endpoints según el verbo HTTP o el camino final al que apuntes.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

GET: Sirve para consultar y leer información sin modificar nada en la base de datos.

POST: Envía datos para crear un nuevo registro o recurso en el servidor.

PUT: Reemplaza por completo un registro existente con los datos nuevos que mandas.

PATCH: Modifica solo uno o varios campos específicos de un registro sin tocar el resto.

DELETE: Borra el recurso indicado en el servidor.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON es un formato de texto plano basado en pares de clave y valor para enviar datos estructurados. Es el estándar porque es sumamente ligero, cualquier lenguaje de programación lo procesa de forma nativa sin complicaciones y resulta muy fácil de leer y depurar cuando estás programando.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es una respuesta numérica de tres dígitos que el servidor te devuelve para decirte qué pasó con tu petición. 200 OK: La petición se resolvió con éxito y el servidor te manda la respuesta esperada. 404 Not Found: La ruta no existe o el registro que buscabas no se encontró en el servidor. 500 Internal Server Error: Ocurrió una falla inesperada o tronó el código del servidor al procesar la solicitud.



7. ¿Qué información se puede encontrar en la respuesta de una API?

Código número y sus texto descriptivo, Metadatos de la información que se envió como el tipo de contenido, y en el body el cuerpo los datos estructurados en el JSON.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Te permite definir el contrato de datos antes de tirar una sola línea de código en el backend. Puedes levantar servidores simulados para que el frontend empiece a trabajar de inmediato con respuestas ficticias y validar que las rutas y estructuras de datos tengan sentido antes de implementarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Es una herramienta visual que genera una página web interactiva con la documentación de tu API. Sirve para ver todos los endpoints disponibles con sus parámetros y para probar peticiones directamente desde el navegador sin necesidad de herramientas externas.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman funciona como un cliente HTTP avanzado para depurar y automatizar pruebas. Te permite armar y mandar peticiones con headers y autenticaciones complejas, guardar colecciones de rutas organizadas por proyecto, cambiar entre entornos de desarrollo o producción rápidamente y correr scripts de prueba para verificar que el servidor responda correctamente.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.